



PROJE TASARIM RAPORU

Ders: BİL204 / Yazılım Mühendisliği

Bölüm/Şube: Bilgisayar Mühendisliği

Grup Adı: Denem

Proje Adı: Proje OBS

Teslim Tarihi: 04.05.2026

Grup Üyeleri:

- Z. Esat Saygın (25100011473)
- Yakup Çaktı (25100011018)
- Muhammed Şerbetçioğlu (24100011080)
- Yaşar A. Yarayan (24100011016)
- Muhammed Barbin (24100011056)
- Tahir Allahverdiyev (24100011822)

İçindekiler

İçindekiler.....	2
1. Giriş.....	4
1.1. Sistemin Amacı.....	4
1.2. Tasarım Hedefleri.....	4
1.2.1. Güvenlik ve Veri Gizliliği (Security & Privacy).....	4
1.2.2. Bakım Yapılabilirlik ve Modülerlik (Maintainability & Modularity).....	4
1.2.3. Kullanılabilirlik (Usability).....	4
2. Üst Düzey Yazılım Mimarisi.....	4
2.1. Alt Sistem Ayrıştırması.....	5
2.1.1. Presentation Layer — React SPA.....	6
2.1.2. Application Layer — Django REST Framework.....	6
2.1.3. Data Layer — Django ORM + SQLite.....	7
2.1.4. Katmanlar Arası İlişkiler.....	7
2.2. Donanım/Yazılım Eşlemesi.....	8
2.3. Kalıcı Veri Yönetimi.....	8
Varlık Tanımları.....	9
Varlıklar Arası İlişkiler.....	10
2.4. Erişim Kontrolü ve Güvenlik.....	11
2.4.1. Kimlik Doğrulama (Authentication).....	11
2.4.2. Yetkilendirme (Authorization).....	12
2.4.3. Güvenlik Önlemleri.....	12
2.4.4. Veri Güvenliği ve Bütünlük.....	12
2.5. Küresel Kontrol Akışı.....	12
2.6. Sınır Koşulları.....	13
2.6.1. Başlatma.....	13
2.6.2. Sonlandırma.....	14
2.6.3. Hata Durumları.....	14
3. Alt Düzey Tasarım.....	15
3.1. Nesne Tasarım Kararları.....	15
3.2. Son Nesne Tasarımı.....	16
3.3. Paketler / Katmanlar.....	17
3.3.1. Dış Paketler.....	17
3.3.2. İç Paketler.....	17
3.4. Sınıf Arayüzleri.....	18
3.5. Tasarım Desenleri.....	26
3.5.1. Facade Pattern — Service Layer (Backend).....	26
3.5.2. Proxy Pattern — Django ORM (Backend).....	26
3.5.3. Strategy Pattern — Django Serializers (Backend).....	26

3.5.4. Template Method Pattern — Django ViewSets (Backend).....	27
3.5.5. Observer Pattern — React State Management (Frontend).....	27
3.5.6. Singleton Pattern — Django Settings & React API Client.....	27
Sözlük (Glossary).....	29
Referanslar.....	30

1. Giriş

Bu bölümde projenin amacı ve temel tasarım hedefleri tanıtılır.

1.1. Sistemin Amacı

Proje OBS, eğitim kurumlarındaki ders kaydı, not girişi ve transkript takibi gibi akademik süreçlerin dijital ortamda merkezi ve güvenli bir şekilde yürütülmesini sağlayan bir web uygulamasıdır. Geleneksel yöntemlerin yarattığı zaman kaybını ve olası insan hatalarını ortadan kaldırmayı hedefler.

Sistemin hedef kullanıcıları ve temel işlemleri şunlardır:

- **Öğrenciler:** Ders seçimi yapma, aktif dönem notlarını ve transkriptlerini (PDF) görüntüleme.
- **Akademisyenler:** Derslerine kayıtlı öğrenci listelerini görüntüleme, not girişi/güncellemesi yapma ve kayıt onaylama.
- **Yöneticiler:** Sistemdeki kullanıcı, bölüm, ders ve dönem tanımlamalarını merkezi admin paneli üzerinden yönetme.

1.2. Tasarım Hedefleri

Projenin mimari kararları alınırken aşağıdaki kalite hedefleri ön planda tutulmuştur:

1.2.1. Güvenlik ve Veri Gizliliği (Security & Privacy)

Kişisel bilgilerin ve notların işlendiği sistemde güvenlik en yüksek önceliktir. Bu hedef; JSON Web Token (JWT) tabanlı kimlik doğrulama, Rol Bazlı Erişim Kontrolü (RBAC) ve veri tabanı saldırılarına (örn. SQL Injection) karşı Django ORM kullanımı ile sağlanmıştır [1][2][3].

1.2.2. Bakım Yapılabilirlik ve Modülerlik (Maintainability & Modularity)

Sistemin ilerleyen süreçlerde kolayca güncellenebilmesi hedeflenmiştir. "Sorumlulukların Ayrılığı" prensibiyle Sunum Katmanı (React SPA) ve Uygulama Katmanı (Django REST) tamamen ayrıştırılmıştır [4][5]. Backend tarafında ise iş mantığı (Service Layer) kontrol katmanından izole edilerek modülerlik ve kodun yeniden kullanılabilirliği artırılmıştır.

1.2.3. Kullanılabilirlik (Usability)

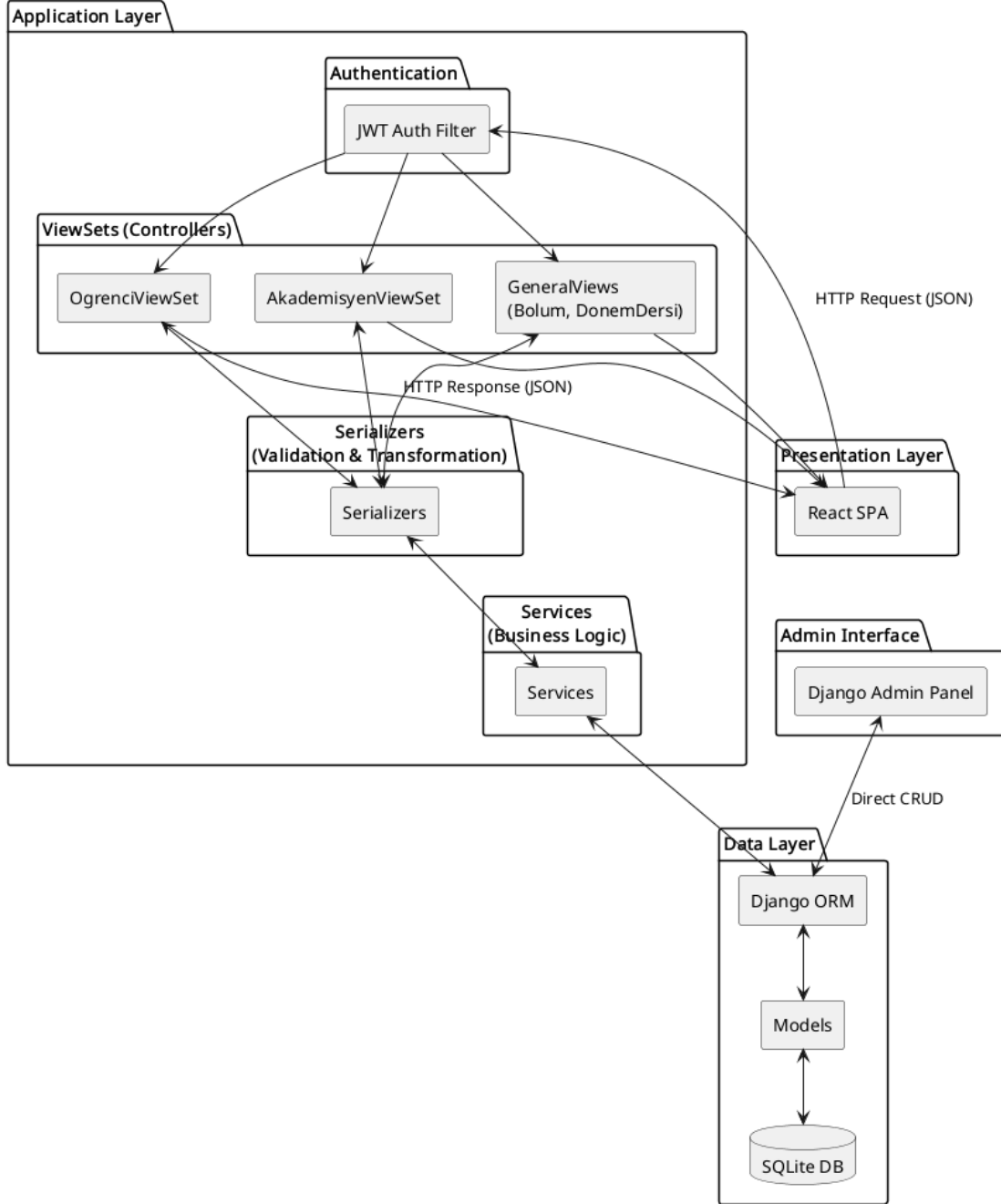
Kullanıcıların sistemi herhangi bir eğitime ihtiyaç duymadan, sezgisel olarak kullanabilmesi amaçlanmıştır. Bu doğrultuda arayüz, React ile Single Page Application (SPA) mimarisinde tasarlanmıştır; sayfa yenilemeleri ortadan kaldırılarak hızlı ve akıcı bir kullanıcı deneyimi sunulmuştur.

2. Üst Düzey Yazılım Mimarisi

Bu bölümde sistemin mimari yapısı, alt sistemlere ayrıştırma, donanım/yazılım eşlemesi, veri yönetimi, erişim kontrolü, kontrol akışı ve sınır koşulları ele alınır.

2.1. Alt Sistem Ayrıştırması

Sistemi oluşturan alt sistemler ve aralarındaki ilişkiler açıklanır.



Sistem, analiz dokümanında tanımlanan gereksinimler doğrultusunda **3 katmanlı (Three-Tier)** ve **decoupled REST mimarisi** kullanılarak tasarlanmıştır [6]. Bu mimaride

sunum katmanı (frontend) ile uygulama katmanı (backend) birbirinden tamamen ayrıştırılmış olup, aralarındaki iletişim yalnızca HTTP protokolü üzerinden JSON formatında REST API aracılığıyla gerçekleştirilmektedir.

2.1.1. Presentation Layer — React SPA

Bu katman, sistemin kullanıcı arayüzünü sağlar ve son kullanıcı ile doğrudan etkileşim halindedir. Öğrenci ve akademisyen rollerine ait tüm ekranlar bu katmanda yer almaktadır. React kullanılarak geliştirilen uygulama, Single Page Application (SPA) mimarisine sahiptir ve sayfa yenileme olmaksızın dinamik içerik sunar [4].

Frontend katmanı, backend ile yalnızca REST API üzerinden iletişim kurar; doğrudan iş mantığına veya veritabanına erişimi bulunmamaktadır. Bu sayede frontend ve backend bileşenleri birbirinden bağımsız olarak geliştirilebilir ve dağıtılabilir.

Buna ek olarak, yönetici (idari personel) işlevleri React tabanlı arayüz üzerinden değil, Django framework'ünün sağladığı **Django Admin paneli** aracılığıyla gerçekleştirilmektedir [3]. Django Admin paneli, kullanıcı arayüzü sağlamakla birlikte doğrudan Django ORM üzerinden veri katmanına erişim sağlar ve REST API katmanını kullanmaz. Bu nedenle yönetici işlemleri, sistemde alternatif bir kullanıcı arayüzü ve erişim noktası olarak modellenmiştir.

2.1.2. Application Layer — Django REST Framework

Bu katman, sistemin iş mantığının yürütüldüğü ana katmandır ve gelen HTTP isteklerinin işlenmesinden sorumludur [5]. Katman, aşağıdaki alt bileşenlerden oluşmaktadır:

JWT Authentication (Middleware):

Sisteme gelen her HTTP isteği öncelikle bu bileşenden geçer. İstek içerisinde bulunan JWT (JSON Web Token) doğrulanır; geçersiz veya süresi dolmuş token durumunda istek reddedilir ve 401 (Unauthorized) yanıtı döndürülür [1]. Geçerli token durumunda kullanıcı kimliği ve rol bilgisi belirlenerek istek bir sonraki bileşene iletilir.

ViewSet (Controller Katmanı):

İsteklerin karşılandığı ve yönlendirildiği bileşendir. HTTP metoduna (GET, POST, PUT, DELETE) ve kullanıcı rolüne göre uygun işlev çağrılır. Sistem kapsamında öğrenci ve akademisyen kullanıcı rolleri için ayrı ViewSet yapıları tanımlanmıştır (OğrenciViewSet, AkademisyenViewSet). Yetkisiz erişim durumlarında 403 (Forbidden) yanıtı döndürülür.

Services (İş Mantığı Katmanı):

İş kurallarının ve uygulama mantığının merkezi olarak yürütüldüğü bileşendir. Ders kaydı, not hesaplama, ders onaylama gibi işlemler bu katmanda gerçekleştirilir. Bu yapı

sayesinde kontrol katmanı (ViewSets) ile iş mantığı ayrıştırılarak daha modüler ve sürdürülebilir bir yapı elde edilmiştir.

Serializers (Veri Dönüşüm ve Doğrulama Katmanı):

İstemci ile sunucu arasında veri dönüşümünü sağlar. JSON formatındaki veriler Django model nesnelere, model nesneleri ise JSON formatına dönüştürülür. Ayrıca veri doğrulama (validation) işlemleri bu katmanda gerçekleştirilir; geçersiz veri durumunda 400 (Bad Request) yanıtı döndürülür.

2.1.3. Data Layer — Django ORM + SQLite

Bu katman, sistemdeki kalıcı verilerin yönetildiği katmandır. Veritabanı işlemleri Django ORM aracılığıyla gerçekleştirilir. ORM kullanımı sayesinde veritabanı işlemleri nesne yönelimli bir şekilde yönetilmekte ve veritabanı bağımlılığı azaltılmaktadır.

Prototip niteliğindeki bu proje için SQLite veritabanı tercih edilmiştir [7]. SQLite, ayrı bir veritabanı sunucusu gerektirmemesi ve Django ile entegre çalışması nedeniyle geliştirme sürecinde kolaylık sağlamaktadır.

Bu katmanda saklanan başlıca veriler şunlardır:

- Kullanıcı bilgileri (öğrenci, akademisyen, yönetici)
- Ders ve dönem bilgileri
- Ders kayıtları ve not bilgileri
- Transkript verileri
- Ders kayıt dönemi bilgileri

2.1.4. Katmanlar Arası İlişkiler

Sistem katmanları arasında tek yönlü ve kontrollü bir bağımlılık bulunmaktadır. Presentation Layer yalnızca Application Layer ile iletişim kurarken, Application Layer veri erişimi için Data Layer'ı kullanır. Data Layer ise üst katmanlardan bağımsızdır ve doğrudan erişime kapalıdır.

Yönetici işlemleri ise bu akıştan farklı olarak Django Admin paneli üzerinden gerçekleştirilmekte ve doğrudan veri katmanına erişmektedir. Bu yapı, sistemde REST tabanlı erişime ek olarak alternatif bir yönetim kanalı sunmaktadır.

Bu mimari yaklaşım sayesinde sistemde düşük bağımlılık (loose coupling), yüksek modülerlik ve bakım yapılabilirlik sağlanmıştır.

2.2. Donanım/Yazılım Eşlemesi

Frontend ve backend bileşenleri aynı geliştirme ortamında barındırılmakla birlikte, birbirinden bağımsız süreçler olarak çalışır.

Yazılım bileşenleri aşağıdaki şekilde eşlenmiştir:

- Frontend (React SPA), Node.js tabanlı geliştirme sunucusu aracılığıyla çalışır ve tarayıcıda istemci tarafında işlenir.
- Backend (Django REST Framework), Python çalışma ortamı üzerinde bir WSGI sunucusu (varsayılan olarak Django'nun geliştirme sunucusu) aracılığıyla çalışır.
- Veritabanı olarak SQLite kullanılmakta olup herhangi bir harici veritabanı sunucusu gerektirmez; SQLite dosyası Django uygulama dizininde yerel olarak saklanır.
- Django Admin paneli, ek bir kurulum gerektirmeksizin Django'nun yerleşik bileşeni olarak aynı süreçte çalışır.

İşletim sistemi bağımsızlığı açısından sistem; Windows, macOS ve Linux ortamlarında çalışabilir. Dağıtım mimarisi aşağıdaki tabloda özetlenmiştir:

Bileşen	Teknoloji	Çalışma Ortamı	Port
Frontend	React + Vite	Node.js	5173
Backend API	Django REST Framework	Python / WSGI	8000
Admin Paneli	Django Admin	Python / WSGI	8000
Veritabanı	SQLite	Dosya sistemi	—

2.3. Kalıcı Veri Yönetimi

Sistem verilerinin kalıcı olarak saklanması amacıyla SQLite veritabanı tercih edilmiştir. Bu tercih, projenin prototip niteliği göz önünde bulundurularak yapılmış olup aşağıdaki avantajlar nedeniyle uygun görülmüştür:

- Ayrı bir veritabanı sunucusu gerektirmemesi
- Django ile sıfır konfigürasyonlu entegrasyon sağlaması
- Geliştirme sürecinde kolay taşınabilirlik sunması

Gerçek üretim ortamına geçişte PostgreSQL gibi daha ölçeklenebilir bir veritabanına geçiş, Django ORM'nin soyutlama katmanı sayesinde minimum değişikliklerle gerçekleştirilebilir.

Veri erişimi Django ORM aracılığıyla nesne yönelimli bir yaklaşımla sağlanmakta; ham SQL sorgusu yazmak yerine Python model sınıfları üzerinden işlem yapılmaktadır. Bu yaklaşım, veri katmanının bakım yapılabilirliğini artırmakta ve veritabanı bağımsızlığını desteklemektedir.

Varlık Tanımları

Sistemde kalıcı olarak saklanan başlıca varlıklar ve alanları aşağıdaki tabloda gösterilmektedir:

Varlık	Alan	Açıklama
KULLANICI (User)	id, username, sifre, email, ad, soyad, role	Tüm kullanıcıların temel kimlik bilgilerini tutar. 'role' alanı Öğrenci / Akademisyen / Yönetici ayrımını sağlar.
OGRENCI	id, kullanıcı_id (FK), bolum_id (FK), ogr_no, sınıf, gpa	Öğrenciye özgü bilgileri saklar. KULLANICI ile 1-1, BOLUM ile N-1 ilişkilidir.
AKADEMISYEN	id, kullanıcı_id (FK), unvan, bolum_id (FK)	Akademisyene özgü bilgileri saklar. KULLANICI ile 1-1, BOLUM ile N-1 ilişkilidir.
BOLUM	id, ad, bolum_kodu	Üniversitedeki akademik bölümleri temsil eder. Öğrenci, Akademisyen ve Ders varlıklarıyla ilişkilidir.

DERS	id, bolum_id (FK), ders_kodu, ad, kredi, min_sinif	Müfredattaki ders tanımlarını tutar. BOLUM ile N-1 ilişkilidir. DonemDersi tarafından referans alınır.
DONEM_DERSI	id, ders_id (FK), akademisyen_id (FK), yil, donem, kontenjan, aktiflik_durumu	Bir dersin belirli bir akademik yıl ve dönemdeki somut açılışını temsil eder. DERS ve AKADEMİSYEN ile N-1 ilişkilidir.
DERS_KAYDI	id, ogrenci_id (FK), donem_dersi_id (FK), vize_notu, final_notu, ortalama, harf_notu, onay_durumu	Öğrenci ile DonemDersi arasındaki ilişkiyi tutar. Çoka-çok ilişkiyi çözen köprü tablodur; not ve onay bilgilerini barındırır.
DERS_KAYIT_DONEMI	id, yıl, donem, baslangic, bitis	Ders kayıt işlemlerinin hangi tarih aralığında yapılabileceğini belirler.

Varlıklar Arası İlişkiler

Varlıklar arasındaki ilişkiler aşağıdaki tabloda özetlenmiştir:

Varlık 1	Varlık 2	İlişki
KULLANICI	OGRENCI	1-1 (her kullanıcının en fazla bir öğrenci kaydı vardır)
KULLANICI	AKADEMİSYEN	1-1 (her kullanıcının en fazla bir akademisyen kaydı vardır)
BOLUM	OGRENCI	1-N (bir bölümde birden fazla öğrenci bulunabilir)
BOLUM	AKADEMİSYEN	1-N (bir bölümde birden fazla akademisyen bulunabilir)
BOLUM	DERS	1-N (bir bölümde birden fazla ders tanımı olabilir)

DERS	DONEM_DERSI	1-N (bir ders birden fazla dönemde açılabilir)
AKADEMISYEN	DONEM_DERSI	1-N (bir akademisyen birden fazla dönem dersi verebilir)
DONEM_DERSI	DERS_KAYDI	1-N (bir dönem dersine birden fazla öğrenci kaydolabilir)
OGRENCI	DERS_KAYDI	1-N (bir öğrencinin birden fazla ders kaydı olabilir)

DERS_KAYDI varlığı, öğrenci ile dönem dersi arasındaki çoka-çok ilişkiyi çözen köprü tablodur ve not bilgilerini (vize, final, ortalama, harf notu) ile onay durumunu bünyesinde barındırır. DONEM_DERSI varlığı, aynı dersin farklı dönemlerde farklı akademisyenler tarafından verilebilmesini modellemektedir. KULLANICI varlığı ise OGRENCI ve AKADEMISYEN tablolarının temel kimlik doğrulama kaydını tutan üst varlıktır; bu yapı Django'nun AbstractUser kalıtım mekanizmasına karşılık gelir.

2.4. Erişim Kontrolü ve Güvenlik

Sistem, kullanıcıların yetkilerine göre erişim sağlayabildiği **rol tabanlı (Role-Based Access Control - RBAC)** bir güvenlik modeli üzerine tasarlanmıştır [7]. Sistemde üç temel kullanıcı rolü bulunmaktadır: Öğrenci, Akademisyen ve Yönetici. Her rol yalnızca kendisine tanımlı işlemleri gerçekleştirebilir.

2.4.1. Kimlik Doğrulama (Authentication)

Sistemde kimlik doğrulama mekanizması olarak **JWT (JSON Web Token)** kullanılmaktadır. Kullanıcı sisteme giriş yaptıktan sonra kendisine bir token üretilir ve bu token sonraki tüm HTTP isteklerinde doğrulama amacıyla kullanılır.

JWT doğrulama süreci aşağıdaki şekilde işler:

- Kullanıcı login olduğunda sistem tarafından JWT token üretilir.
- Bu token frontend tarafından saklanır ve her istekte Authorization header içerisinde gönderilir.
- Backend tarafında JWT Auth Middleware, gelen istekteki token'ı doğrular.
- Token geçersiz veya süresi dolmuş ise istek reddedilir ve **401 Unauthorized** hatası döndürülür.

2.4.2. Yetkilendirme (Authorization)

Kimlik doğrulaması başarılı olan kullanıcılar için rol bazlı yetkilendirme yapılır. Kullanıcının rol bilgisi JWT içerisinde elde edilerek erişim kontrolü sağlanır.

- **Öğrenci:** Ders seçme, kayıt görüntüleme, transkript görüntüleme ve PDF indirme işlemlerini gerçekleştirebilir.
- **Akademisyen:** Derslere ait öğrenci listelerini görüntüleyebilir, not girişi ve güncellemesi yapabilir, ders kayıt taleplerini onaylayabilir.
- **Yönetici:** Sistem yönetimi işlemlerini Django Admin paneli üzerinden gerçekleştirir (kullanıcı, ders ve dönem yönetimi).

Yetkisiz erişim durumlarında sistem **403 Forbidden** yanıtı döndürerek işlemi engeller.

2.4.3. Güvenlik Önlemleri

Sistemde veri güvenliğini sağlamak amacıyla aşağıdaki önlemler uygulanmaktadır:

- **JWT tabanlı stateless authentication** kullanımı
- Şifrelerin Django'nun built-in password hashing mekanizması ile güvenli şekilde saklanması
- Rol bazlı erişim kontrolü (RBAC)
- Backend tarafında tüm kritik işlemlerin doğrulama katmanından geçirilmesi
- REST API endpoint'lerinin yalnızca yetkili kullanıcılar tarafından erişilebilir olması

2.4.4. Veri Güvenliği ve Bütünlük

Veri bütünlüğünü korumak amacıyla:

- Tüm veri girişleri serializer katmanında doğrulanmaktadır.
- Geçersiz veya eksik veri sistem tarafından kabul edilmez.
- Veritabanı işlemleri Django ORM üzerinden gerçekleştirilerek SQL injection riskleri minimize edilmiştir.

2.5. Küresel Kontrol Akışı

Sistemde kontrol akışı istemci-sunucu mimarisi üzerine kurulmuş REST tabanlı bir yapı ile sağlanmaktadır. Kullanıcı tarafından gerçekleştirilen işlemler, React tabanlı frontend üzerinden HTTP istekleri aracılığıyla backend sistemine iletilir.

Backend tarafında gelen istekler sırasıyla authentication (JWT doğrulama), yetkilendirme (rol kontrolü) ve ViewSet katmanından geçerek işlenir. Geçerli istekler Service katmanına yönlendirilir ve iş kuralları burada uygulanır. Service katmanı, veri işlemlerini Django ORM aracılığıyla veri katmanını üzerinde gerçekleştirir. İşlem sonuçları Serializer katmanını ile JSON formatına dönüştürülerek HTTP response olarak istemciye geri gönderilir.

Sistem genel olarak senkron (synchronous) request-response modeli kullanmaktadır. Her istek tamamlanmadan istemciye yanıt dönülmemektedir. Kontrol mekanizması merkezi olarak backend katmanında toplanmıştır; tüm iş mantığı ve veri erişimi bu katman üzerinden yürütülmektedir.

Alt sistemler arası koordinasyon REST API ve ORM aracılığıyla sağlanmaktadır. Frontend ile backend arasındaki iletişim HTTP/JSON tabanlı REST API üzerinden gerçekleşirken, backend ile veritabanı arasındaki iletişim Django ORM aracılığıyla sağlanmaktadır. Yönetici işlemleri ise Django Admin paneli üzerinden doğrudan veri katmanına erişerek gerçekleştirilir.

2.6. Sınır Koşulları

React SPA + Django REST Framework + SQLite + JWT tabanlı 3 katmanlı mimariye göre projenin başlatma, sonlandırma ve hata sınır koşulları şu şekilde çalışacaktır:

2.6.1. Başlatma

Sistemin hem frontend hem de backend tarafında ayağa kalkıp hizmet verebilir duruma gelmesi için gereken koşullar:

- **Backend Başlatma:** Django sunucusunun ayağa kalkması, settings.py içerisindeki global ayarların yüklenmesi ve SQLite veritabanı bağlantısının sorunsuz kurulması.
- **Frontend Başlatma:** Tarayıcıda index.html üzerinden uygulamanın yüklenmesi, ThemeContext ve i18n yapılandırmalarının localStorage dan okunarak UI' nin dil ve tema ayarlarıyla render edilmesi. Ortam değişkenlerinden (VITE_API_BASE_URL) backend adresinin tanınması.
- **Authentication Başlatma:** Uygulama açıldığında ilk olarak localStorage içinde bir JWT token olup olmadığı kontrol edilir.

> *token yoksa:* Kullanıcı varsayılan sınır olan LoginHomePage ekranına yönlendirilir.

> *token varsa:* Token'ın payload kısmındaki "rol" (Öğrenci veya Akademisyen) bilgisi okunur ve kullanıcı doğrudan kendi paneline (/student veya /academician) yönlendirilir.

2.6.2. Sonlandırma

Bir web uygulaması olduğu için sunucu sürekli çalışır (stateless), ancak kullanıcı döngüsünün sonlanması belirli sınırlara tabidir:

- **İstemci Bellek Sonlanması:** Kullanıcının tarayıcı sekmesini kapatmasıyla birlikte React'in bellekte tuttuğu tüm geçici veriler (state) yok olur. Yalnızca JWT token, dil ve tema tercihleri localStorage'da kalır.
- **Aktif Oturumun Logout işlemi:** Kullanıcı sistemden çıkış yapmak istediğinde, AuthContext içerisindeki çıkış fonksiyonu tetiklenir. JWT token ve rol bilgisi localStorage'dan tamamen silinir ve kullanıcı tekrar Login ekranına atılır.
- **Pasif Oturum Sonlanması:** Sistemin güvenliği için JWT token'larının bir yaşam süresi vardır. Bu süre dolduğunda, kullanıcı hiçbir çıkış işlemi yapmasa bile backend gelen isteklere 401 Unauthorized döner ve sistem oturumu otomatik olarak sonlandırarak kullanıcıyı dışarı atar.

2.6.3. Hata Durumları

Sistemin beklenmeyen durumlarda, geçersiz verilerde veya yetki aşımalarında vereceği tepkiler:

- **Kimlik Doğrulama Hataları (401 Unauthorized):** JWT Auth Filter (Middleware), gelen HTTP isteğindeki token'ı doğrulayamazsa (geçersiz, bozuk veya süresi dolmuşsa), istek iş mantığına (Service Layer) hiç ulaşmadan 401 hatasıyla reddedilir. Frontend bu hatayı yakalayarak kullanıcıyı giriş yapmaya zorlar.
- **Yetki İhlali Hataları (403 Forbidden):** Role-Based Access Control sınırlarına göre; örneğin bir öğrenci, akademisyene ait bir endpoint'e (not girme/güncelleme) istek atarsa sistem 403 Forbidden döner ve işlemi engeller.
- **Veri Doğrulama Hataları (400 Bad Request):** İstemciden gelen eksik veya hatalı JSON verileri (örneğin 100 üzerinden 150 not girilmesi), backend'deki *Serializers* katmanında doğrulamaya takılır ve veritabanına yazılmadan 400 Bad Request yanıtıyla reddedilir.
- **İş Mantığı İhlalleri (Service Layer Errors):** Tasarım belgesinde belirtilen "Ders Kayıt Dönemi Kontrolü" gibi kuralların ihlali durumunda (örneğin kayıt dönemi dışında ders seçmeye çalışmak), EnrollmentService işlemi engeller ve frontend'e anlamlı bir hata mesajı döndürür (Örn: "Ders kayıt dönemi dışındasınız").
- **Arayüz Yönlendirme Hataları (404 Not Found):** React Router'da tanımlı olmayan bir sayfa yoluna gidilirse sistem çökmez, <NotFound/> bileşenini render ederek sınır ihlalini absorbe eder.
- **Sistem/Sunucu Çökmeleri (500 Internal Server Error):** SQLite veritabanının kilitlenmesi veya backend sunucusunda bir exception oluşması durumunda,

frontend tarafında kullanıcıya teknik bir hata yerine tr.json'da tanımladığınız Beklenmeyen bir hata oluştu mesajı gösterilir.

3. Alt Düzey Tasarım

Bu bölümde sistemin sınıf düzeyindeki tasarımı, tasarım kararları, paketler, sınıf arayüzleri ve kullanılan tasarım desenleri detaylandırılır.

3.1. Nesne Tasarım Kararları

Analiz aşamasında iş kuralları nesne yönelimli yaklaşım doğrultusunda model sınıflarının metotları içerisinde (örn. `ders_kaydi_yap()`, `not_gir_guncelle()`) tanımlanmıştır. Ancak tasarım aşamasında sistemin bakım yapılabirliğini ve modülerliğini artırmak amacıyla bu iş kuralları Service Layer içerisine taşınmıştır.

Bu değişiklik ile:

- İş mantığı controller (ViewSet) ve veri modeli (Model) katmanlarından ayrıştırılmıştır
- Aynı iş kurallarının farklı endpoint'lerde tekrar kullanılabilmesi sağlanmıştır
- Kodun test edilebilirliği ve sürdürülebilirliği artırılmıştır

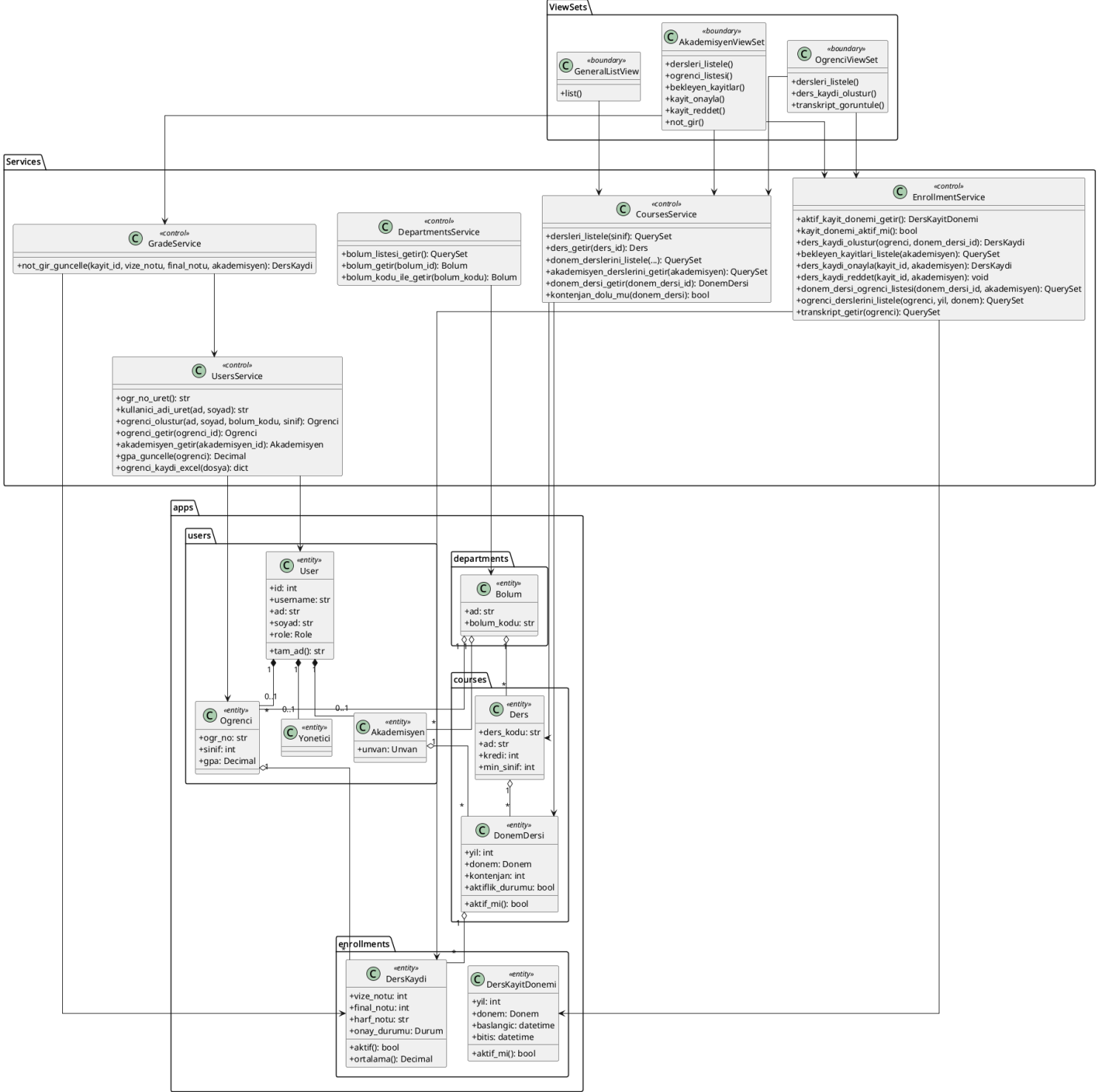
Bu yaklaşım, analiz aşamasındaki kavramsal modelin daha ölçeklenebilir bir mimariye dönüştürülmesi olarak değerlendirilmelidir.

Frontend ve backend katmanları tamamen bağımsız olacak şekilde tasarlanmış, React tabanlı kullanıcı arayüzü yalnızca REST API üzerinden backend ile haberleşmiştir. Bu yaklaşım sayesinde sistemin dağıtık bir yapıda çalışabilmesi ve farklı istemcilerin (web, mobil vb.) aynı backend'i kullanabilmesi mümkün hale getirilmiştir.

Veritabanı olarak SQLite tercih edilmiştir çünkü proje geliştirme sürecinde kurulum gerektirmeyen, hafif ve hızlı bir çözüm sunmaktadır. ORM kullanımı sayesinde veritabanı bağımlılığı azaltılmış ve ileride farklı bir veritabanına geçiş kolaylaştırılmıştır.

Authentication mekanizması olarak JWT tercih edilmiştir. Bunun nedeni stateless yapısı sayesinde sunucu tarafında oturum yönetimi ihtiyacını ortadan kaldırması ve ölçeklenebilirliği artırmasıdır.

3.2. Son Nesne Tasarımı



3.3. Paketler / Katmanlar

3.3.1. Dış Paketler

Sistemin geliştirilmesinde kullanılan başlıca dış kütüphaneler ve framework'ler aşağıda listelenmiştir:

React

Kullanıcı arayüzünün geliştirilmesi için kullanılan JavaScript kütüphanesidir. Bileşen tabanlı yapısı sayesinde tekrar kullanılabilir UI elemanları oluşturulmuş ve Single Page Application (SPA) mimarisi uygulanmıştır.

Vite

React uygulamasının geliştirme ve derleme süreçlerini yöneten araçtır. Geliştirme sunucusu sağlar ve frontend uygulamasının hızlı bir şekilde çalıştırılmasına olanak tanır.

Django

Backend uygulamasının temelini oluşturan Python web framework'üdür. Model, view ve yönetim paneli gibi temel bileşenleri sağlar.

Django REST Framework (DRF)

Django üzerinde RESTful API geliştirmek için kullanılan eklentidir. ViewSet, Serializer ve authentication mekanizmaları ile API geliştirme sürecini kolaylaştırır.

djangorestframework-simplejwt

JWT tabanlı kimlik doğrulama işlemlerini gerçekleştirmek için kullanılır. Kullanıcı giriş işlemleri sonrası token üretimi ve doğrulama işlemlerini sağlar.

SQLite

Projenin geliştirme aşamasında kullanılan hafif ve dosya tabanlı veritabanıdır. Ek bir kurulum gerektirmemesi nedeniyle tercih edilmiştir.

3.3.2. İç Paketler

Sistem, Django'nun "app" mimarisi kullanılarak modüler şekilde tasarlanmıştır:

- **users**

Kullanıcı yönetimi ile ilgilidir. User, Öğrenci, Akademisyen ve Yönetici modellerini içerir. Ayrıca rol bazlı yetkilendirme (permissions) mekanizmaları bu pakette tanımlanmıştır.

- **courses**
Ders ve dönem dersi (DonemDersi) varlıklarını içerir. Ders tanımları ve akademisyenlere atanmış dönem dersleri bu paket üzerinden yönetilir.
- **enrollments**
Ders kayıt işlemleri ile ilgilidir. DersKaydi ve DersKayitDonemi modellerini içerir. Ders seçimi, onay süreçleri ve not hesaplama gibi işlemler bu modül kapsamında yürütülür.
- **departments**
Bölüm (Bolum) varlığını içerir. Öğrenci ve akademisyenlerin bağlı olduğu akademik birimleri temsil eder.
- **authentication**
JWT tabanlı kimlik doğrulama işlemlerini içerir. Token üretimi ve doğrulama süreçleri bu paket üzerinden yönetilmektedir.

3.4. Sınıf Arayüzleri

Her bir önemli sınıfın (entity, control, boundary) öznitelikleri ve metotları detaylı olarak açıklanır. Her metot için imza (parametreler, dönüş değeri) ve kısa bir açıklama verilir.

ENTITY SINIFLARI

C-1: User (Kullanıcı) Sınıfı

apps.users | <<entity>>

Amaç

Sistemdeki tüm kullanıcı tipleri (Öğrenci, Akademisyen, Yönetici) için ortak temel kullanıcı yapısını temsil eder. Django'nun `AbstractUser` mekanizmasına karşılık gelir.

Öznitelikler

- `id: int` — Benzersiz kullanıcı kimliği
- `username: str` — Kullanıcı adı (giriş için kullanılır)
- `ad: str` — Kullanıcının adı
- `soyad: str` — Kullanıcının soyadı
- `role: Role` — Kullanıcı rolü (Öğrenci / Akademisyen / Yönetici)

Metotlar

- `tam_ad() -> str` — Kullanıcının ad ve soyadını birleştirerek tam adını döndürür.

İlişkiler

- Öğrenci, Akademisyen ve Yönetici sınıflarının üst (parent) sınıfıdır (Composition, 1–0..1).

C-2: Öğrenci Sınıfı

apps.users | <<entity>>

Amaç

Sistemdeki öğrenci kullanıcılarına ait akademik bilgileri tutar.

Öznitelikler

- ogr_no: str — Öğrenci numarası
- sinif: int — Öğrencinin bulunduğu sınıf seviyesi
- gpa: Decimal — Genel not ortalaması

İlişkiler

- User sınıfından türetilmiştir (Composition, 1–0..1)
 - Bolum ile Aggregation: 1 Bölüm → * Öğrenci
 - DersKaydi ile Aggregation: 1 Öğrenci → * DersKaydi
-

C-3: Akademisyen Sınıfı

apps.users | <<entity>>

Amaç

Sistemdeki akademisyen kullanıcılarına ait bilgileri tutar.

Öznitelikler

- unvan: Unvan — Akademik unvan (Profesör, Doçent, Dr. Öğr. Üyesi vb.)

İlişkiler

- User sınıfından türetilmiştir (Composition, 1–0..1)
 - Bolum ile Aggregation: 1 Bölüm → * Akademisyen
 - DonemDersi ile Aggregation: 1 Akademisyen → * DonemDersi
-

C-4: Yonetici Sınıfı

apps.users | <<entity>>

Amaç

Sistemin idari işlemlerini yürüten yönetici kullanıcı tipini temsil eder. Tüm yönetim işlemleri Django Admin paneli üzerinden gerçekleştirilir; bu sınıfa ait ayrı bir metot tanımlanmamıştır.

İlişkiler

- User sınıfından türetilmiştir (Composition, 1–0..1)
-

C-5: Bolum Sınıfı

apps.departments | <<entity>>

Amaç

Üniversitedeki akademik bölümleri temsil eder.

Öznitelikler

- ad: str — Bölümün adı
- bolum_kodu: str — Bölümün kısa kodu

İlişkiler

- 1 Bölüm → * Öğrenci (Aggregation)
- 1 Bölüm → * Akademisyen (Aggregation)
- 1 Bölüm → * Ders (Aggregation)

C-6: Ders Sınıfı

apps.courses | <<entity>>

Amaç

Müfredatta yer alan temel ders tanımlarını temsil eder.

Öznitelikler

- ders_kodu: str — Dersin kısa kodu
- ad: str — Dersin adı
- kredi: int — Kredi değeri
- min_sinif: int — Derse kayıt için gereken minimum sınıf seviyesi

İlişkiler

- Bolum ile Aggregation: 1 Bölüm → * Ders
- DonemDersi ile Aggregation: 1 Ders → * DonemDersi

C-7: DonemDersi Sınıfı

apps.courses | <<entity>>

Amaç

Bir dersin belirli bir akademik yıl ve dönemde açılan somut halini temsil eder.

Öznitelikler

- yıl: int — Akademik yıl
- donem: Donem — Dönem bilgisi (Güz / Bahar vb.)
- kontenjan: int — Derse kabul edilebilecek maksimum öğrenci sayısı
- aktiflik_durumu: bool — Dersin aktif olup olmadığı

Metotlar

- aktif_mi() -> bool — Dönem dersinin aktif kayıt için uygun olup olmadığını döndürür.

İlişkiler

- Ders ile Aggregation: DonemDersi → 1 Ders
 - Akademisyen ile Aggregation: DonemDersi → 1 Akademisyen
 - DersKaydi ile Aggregation: 1 DonemDersi → * DersKaydi
-

C-8: DersKaydi Sınıfı

apps.enrollments | <<entity>>

Amaç

Öğrencinin bir dönem dersine kayıt durumunu, not bilgisini ve onay durumunu tutar. Öğrenci ile DonemDersi arasındaki çoka-çok ilişkiyi çözen köprü tablodur.

Öznitelikler

- vize_notu: int — Vize sınavı notu
- final_notu: int — Final sınavı notu
- harf_notu: str — Hesaplanan harf notu
- onay_durumu: Durum — Kaydın onay durumu (Bekliyor / Onaylandı / Reddedildi)

Metotlar

- ortalama() -> Decimal — Vize (%40) ve final (%60) ağırlıklarına göre ağırlıklı ortalamayı hesaplar.
- aktif() -> bool — Kaydın aktif (onaylanmış ve geçerli) olup olmadığını döndürür.

İlişkiler

- Öğrenci ile Aggregation: 1 Öğrenci → * DersKaydi
 - DonemDersi ile Aggregation: DersKaydi → 1 DonemDersi
-

C-9: DersKayitDonemi Sınıfı

apps.enrollments | <<entity>>

Amaç

Ders kayıt işlemlerinin gerçekleştirilebileceği zaman aralığını tanımlar.

Öznitelikler

- yıl: int — Akademik yıl
- donem: Donem — Dönem bilgisi
- baslangic: datetime — Kayıt döneminin başlangıç tarihi ve saati
- bitis: datetime — Kayıt döneminin bitiş tarihi ve saati

Metotlar

- aktif_mi() -> bool — Mevcut tarih ve saatin kayıt dönemi aralığında olup olmadığını kontrol eder.

CONTROL (SERVICE) SINIFLARI

C-10: UsersService Sınıfı

Services | <<control>>

Amaç

Kullanıcı oluşturma, sorgulama ve GPA güncelleme gibi kullanıcı yönetimine ilişkin iş kurallarını merkezi olarak yürütür.

Metotlar

- `ogr_no_uret() -> str` — Yeni öğrenci için benzersiz bir öğrenci numarası üretir.
- `kullanici_adi_uret(ad: str, soyad: str) -> str` — Ad ve soyad bilgisinden sistem genelinde benzersiz bir kullanıcı adı üretir.
- `ogrenci_olustur(ad: str, soyad: str, bolum_kodu: str, sinif: int) -> Ogrenci` — Yeni öğrenci kaydı oluşturur ve ilişkili User nesnesini döndürür.
- `ogrenci_getir(ogrenci_id: int) -> Ogrenci` — Verilen kimliğe sahip öğrenciyi döndürür.
- `akademisyen_getir(akademisyen_id: int) -> Akademisyen` — Verilen kimliğe sahip akademisyeni döndürür.
- `gpa_guncelle(ogrenci: Ogrenci) -> Decimal` — Öğrencinin tüm onaylı ders kayıtlarından GPA'ini yeniden hesaplayıp kaydeder ve güncel değeri döndürür.
- `ogrenci_kaydi_excel(dosya: File) -> dict` — Excel dosyasından toplu öğrenci kaydı oluşturur; işlem sonuç özetini sözlük olarak döndürür.

Bağımlılıklar

- Ogrenci, User entity sınıfları

C-11: CoursesService Sınıfı

Services | <<control>>

Amaç

Ders ve dönem dersi sorgulama ile kontenjan kontrolü gibi ders yönetimine ilişkin iş kurallarını yürütür.

Metotlar

- `dersleri_listele(sinif: int) -> QuerySet` — Verilen sınıf seviyesine uygun (`min_sinif <= sinif`) dersleri döndürür.
- `ders_getir(ders_id: int) -> Ders` — Verilen kimliğe sahip ders nesnesini döndürür.
- `donem_derslerini_listele(...)` -> QuerySet — Belirtilen filtreler (yıl, dönem, bölüm vb.) doğrultusunda dönem derslerini listeler.

- `akademisyen_derslerini_getir(akademisyen: Akademisyen) -> QuerySet` — İlgili akademisyene atanmış dönem derslerini döndürür.
- `donem_dersi_getir(donem_dersi_id: int) -> DonemDersi` — Verilen kimliğe sahip dönem dersi nesnesini döndürür.
- `kontenjan_dolu_mu(donem_dersi: DonemDersi) -> bool` — Dönem dersinin kontenjanının dolup dolmadığını kontrol eder; dolu ise `True` döndürür.

Bağımlılıklar

- `Ders`, `DonemDersi` entity sınıfları

C-12: EnrollmentService Sınıfı

Services | <<control>>

Amaç

Ders kayıt iş akışını (kayıt oluşturma, onaylama, reddetme) ve transkript sorgularını merkezi olarak yürütür.

Metotlar

- `aktif_kayit_donemi_getir() -> DersKayitDonemi` — Mevcut aktif kayıt dönemini döndürür; aktif dönem yoksa hata fırlatır.
- `kayit_donemi_aktif_mi() -> bool` — Şu an bir kayıt dönemi aktif olup olmadığını döndürür.
- `ders_kaydi_olustur(ogrenci: Ogrenci, donem_dersi_id: int) -> DersKaydi` — Kayıt dönemi kontrolü, kontenjan kontrolü ve müfredat uygunluğu doğrulaması yaparak yeni ders kaydı oluşturur.
- `bekleyen_kayitlari_listele(akademisyen: Akademisyen) -> QuerySet` — İlgili akademisyenin derslerine ait onay bekleyen kayıtları döndürür.
- `ders_kaydi_onayla(kayit_id: int, akademisyen: Akademisyen) -> DersKaydi` — Belirtilen kaydı onaylar ve güncellenmiş `DersKaydi` nesnesini döndürür.
- `ders_kaydi_reddet(kayit_id: int, akademisyen: Akademisyen) -> void` — Belirtilen kaydı reddeder.
- `donem_dersi_ogrenci_listesi(donem_dersi_id: int, akademisyen: Akademisyen) -> QuerySet` — Belirli dönem dersine kayıtlı öğrencilerin listesini döndürür.
- `ogrenci_derslerini_listele(ogrenci: Ogrenci, yil: int, donem: Donem) -> QuerySet` — Öğrencinin belirtilen yıl ve döneme ait ders kayıtlarını döndürür.
- `transkript_getir(ogrenci: Ogrenci) -> QuerySet` — Öğrencinin tüm onaylı ders kayıtlarını döndürür.

Bağımlılıklar

- `DersKaydi`, `DersKayitDonemi` entity sınıfları

C-13: GradeService Sınıfı

Services | <<control>>

Amaç

Not girişi ve güncelleme iş kurallarını merkezi olarak yürütür. İşlem sonrasında UsersService üzerinden öğrencinin GPA'ini günceller.

Metotlar

- `not_gir_guncelle(kayit_id: int, vize_notu: int, final_notu: int, akademisyen: Akademisyen) -> DersKaydi` — Belirtilen ders kaydına vize ve final notlarını girer ya da günceller. Yetki doğrulaması yaparak yalnızca ilgili akademisyenin kendi dersi için not girebilmesini sağlar. Notlar kaydedildikten sonra `UsersService.gpa_guncelle()` çağrılarak öğrencinin GPA'i güncellenir. Güncellenmiş `DersKaydi` nesnesini döndürür.

Bağımlılıklar

- `DersKaydi` entity sınıfı
 - `UsersService` (GPA güncelleme için)
-

C-14: DepartmentsService Sınıfı

Services | <<control>>

Amaç

Bölüm sorgulama işlemlerini merkezi olarak yürütür.

Metotlar

- `bolum_listesi_getir() -> QuerySet` — Sistemdeki tüm bölümleri döndürür.
- `bolum_getir(bolum_id: int) -> Bolum` — Verilen kimliğe sahip bölümü döndürür.
- `bolum_kodu_ile_getir(bolum_kodu: str) -> Bolum` — Verilen bölüm koduna sahip bölümü döndürür.

Bağımlılıklar

- `Bolum` entity sınıfı
-

BOUNDARY (VIEWSET) SINIFLARI

C-15: OgrenciViewSet Sınıfı

ViewSets | <<boundary>>

Amaç

Öğrenci rolüne ait HTTP isteklerini karşılayan kontrolördür. JWT doğrulaması ve öğrenci rol kontrolünden geçen istekleri ilgili servis katmanına yönlendirir.

Metotlar

- `dersleri_listele()` -> ResponseGET /ogrenci/dersler/ — Öğrencinin sınıf seviyesine uygun müsait dönem derslerini listeler. `CoursesService.dersleri_listele()` çağrısını kullanır.
- `ders_kaydi_olustur()` -> ResponsePOST /ogrenci/kayit/ — Öğrencinin seçtiği derse kayıt talebini oluşturur. `EnrollmentService.ders_kaydi_olustur()` çağrısını kullanır.
- `transkript_goruntule()` -> ResponseGET /ogrenci/transkript/ — Öğrencinin tüm dönemlere ait onaylı ders kayıtlarını döndürür. `EnrollmentService.transkript_getir()` çağrısını kullanır.

Bağımlılıklar

- `EnrollmentService, CoursesService`

C-16: AkademisyenViewSet Sınıfı

ViewSets | <<boundary>>

Amaç

Akademisyen rolüne ait HTTP isteklerini karşılayan kontrolördür. JWT doğrulaması ve akademisyen rol kontrolünden geçen istekleri ilgili servis katmanına yönlendirir.

Metotlar

- `dersleri_listele()` -> ResponseGET /akademisyen/dersler/ — Akademisyenin sorumlu olduğu dönem derslerini listeler. `CoursesService.akademisyen_derslerini_getir()` çağrısını kullanır.
- `ogrenci_listesi()` -> ResponseGET /akademisyen/dersler/{id}/ogrenciler/ — Belirli dönem dersine kayıtlı öğrenci listesini döndürür. `EnrollmentService.donem_dersi_ogrenci_listesi()` çağrısını kullanır.
- `bekleyen_kayitlar()` -> ResponseGET /akademisyen/kayitlar/bekleyen/ — Onay bekleyen ders kayıtlarını listeler. `EnrollmentService.bekleyen_kayitlari_listele()` çağrısını kullanır.
- `kayit_onayla()` -> ResponsePOST /akademisyen/kayitlar/{id}/onayla/ — Belirtilen ders kaydını onaylar. `EnrollmentService.ders_kaydi_onayla()` çağrısını kullanır.
- `kayit_reddet()` -> ResponsePOST /akademisyen/kayitlar/{id}/reddet/ — Belirtilen ders kaydını reddeder. `EnrollmentService.ders_kaydi_reddet()` çağrısını kullanır.
- `not_gir()` -> ResponsePUT /akademisyen/kayitlar/{id}/not/ — Öğrenci için vize ve final notu girer ya da günceller. `GradeService.not_gir_guncelle()` çağrısını kullanır.

Bağımlılıklar

- `EnrollmentService, GradeService, CoursesService`

3.5. Tasarım Desenleri

Sistemde kod kalitesini ve sürdürülebilirliği artırmak amacıyla hem backend (Django) hem de frontend (React) tarafında GoF (Gang of Four) tasarım desenleri bilinçli olarak uygulanmıştır [8].

3.5.1. Facade Pattern — Service Layer (Backend)

Kullanım Yeri: `EnrollmentService`, `GradeService`, `TranscriptService`

İş mantığı View katmanından (ViewSet) ayrıştırılarak ayrı bir Service katmanında toplanmıştır. Bu sayede karmaşık iş kuralları (ders kayıt dönemi kontrolü, not ortalaması hesaplama, transkript oluşturma gibi) tek bir basit arayüz arkasına gizlenmiş ve ViewSet'ler bu servisleri çağırarak yetinmiştir.

Facade deseninin tercih edilme nedeni **bakım yapılabilirlik** ve **yeniden kullanılabilirlik** tir. İş mantığı merkezileştirildiği için aynı işlem farklı endpoint'lerden çağrılabilir bile kod tekrarı önlenmiştir. Örneğin `ders_kaydi_yap()` metodu birden fazla ViewSet tarafından kullanılabilir ve iş kuralı değişikliği durumunda yalnızca Service katmanında tek bir yerde güncelleme yapılması yeterlidir.

3.5.2. Proxy Pattern — Django ORM (Backend)

Kullanım Yeri: **Django ORM** ve Model sınıfları (`User`, `Ogrenci`, `Akademisyen`, `DersKaydi` vb.)

Django ORM, uygulama ile veritabanı arasında **Proxy** görevi görmektedir. Model sınıfları (`User`, `Ogrenci`, `Akademisyen`) üzerinden yapılan işlemler doğrudan SQL'e dönüştürülmekte, ancak uygulama kodundan SQL detayları soyutlanmaktadır. Bu sayede veritabanı değişikliği (örn. SQLite'tan PostgreSQL'e geçiş) durumunda Service ve ViewSet katmanlarında kod değişikliği yapılmasına gerek kalmamaktadır.

ORM aynı zamanda lazy loading ve query optimization sağlayarak performans kontrolünü de elinde tutmaktadır.

3.5.3. Strategy Pattern — Django Serializers (Backend)

Kullanım Yeri: Serializers (veri doğrulama ve dönüşüm katmanı)

Farklı kullanıcı rolleri ve farklı endpoint'ler için farklı veri doğrulama ve dönüşüm stratejileri uygulanmaktadır. Django REST Framework'ün Serializer yapısı, bu stratejileri runtime'da değiştirilebilir ve modüler olarak tanımlanabilir hale getirir.

Örneğin bir `DersKaydi` nesnesi öğrenci için sadece okuma yetkisiyle serileştirilirken, akademisyen için not girişine izin veren farklı bir serializer kullanılabilir. Bu yaklaşım, farklı kullanım senaryoları için ayrı validation ve transformation logic'i tanımlamayı mümkün kılar.

3.5.4. Template Method Pattern — Django ViewSets (Backend)

Kullanım Yeri: `OgrenciViewSet`, `AkademisyenViewSet`

Django REST Framework'ün `ViewSet` yapısı, HTTP işlemlerinin (GET, POST, PUT, DELETE) genel akışını tanımlar ve bu akışın belirli adımlarını (örn. authentication, permission check, serialization) alt sınıflarda override edilebilir hale getirir.

`OgrenciViewSet` ve `AkademisyenViewSet` sınıfları, temel `ViewSet` akışını kullanırken yetkilendirme ve veri filtreleme adımlarını kendi rollerine göre özelleştirir. Bu sayede ortak işlem akışı korunmuş, yalnızca farklılık gösteren kısımlar özelleştirilmiştir.

3.5.5. Observer Pattern — React State Management (Frontend)

Kullanım Yeri: React Hooks (`useState`, `useEffect`) ve komponent yeniden render mekanizması

React'in temel çalışma prensibi Observer Pattern'e benzer bir mekanizma üzerine kuruludur. Bir state değiştiğinde (örn. kullanıcı ders kayıt listesini güncellediğinde), bu state'i kullanan tüm bileşenler otomatik olarak yeniden render edilir ve UI güncel veri ile senkronize olur.

Bu desen sayesinde frontend'de manuel DOM manipülasyonu ortadan kalkar ve veri ile arayüz arasında otomatik bir bağ kurulur. Örneğin bir öğrenci not listesini görüntülerken, backend'den gelen yeni not verisi state'e yazıldığında tablo bileşeni otomatik olarak güncellenir.

3.5.6. Singleton Pattern — Django Settings & React API Client

Kullanım Yeri:

- Django `settings.py` (Backend)
- Axios instance / API client (Frontend)

Backend: Django'nun ayar yapılandırması uygulama boyunca **tek bir merkezi yapılandırma nesnesi** olarak çalışır. Veritabanı bağlantı bilgileri, JWT secret key, debug modu gibi global ayarlar bu dosyada tanımlanır ve tüm backend bu tek kaynağa başvurur.

Frontend: React uygulamasında API istekleri için kullanılan Axios instance'ı, uygulama başlangıcında bir kez yapılandırılır ve tüm komponentler tarafından import edilerek kullanılır (Singleton Pattern). Base URL, authentication header'ları ve timeout ayarları bu singleton instance üzerinde tanımlanır. Bu sayede her API çağrısında bu ayarların tekrar edilmesine gerek kalmaz ve tutarlılık sağlanır.

Sözlük (Glossary)

Raporda kullanılan teknik terimlerin ve kavramların tanımları bu bölümde verilir.

API: Yazılım bileşenleri arasında veri iletişimini sağlayan arayüzdür.

Backend: İş mantığı ve veri işlemlerinin yürütüldüğü sunucu tarafı katmandır.

Frontend: Kullanıcı ile etkileşimin gerçekleştiği istemci tarafı arayüz katmanıdır.

REST: HTTP protokolü üzerinde çalışan, istemci-sunucu mimarisine dayalı bir web servis yaklaşımıdır.

SPA (Single Page Application): Sayfa yenilemesi olmadan dinamik içerik sunan web uygulama mimarisidir.

JWT (JSON Web Token): Kullanıcı kimlik doğrulamasında kullanılan, imzalı token yapısıdır.

Authentication: Kullanıcının sisteme kimlik doğrulama sürecidir.

Authorization: Kullanıcının yetkilerinin belirlenmesi sürecidir.

ORM (Object Relational Mapping): Veritabanı işlemlerinin nesne yönelimli yapı üzerinden gerçekleştirilmesini sağlayan katmandır.

Service Layer: İş mantığının controller katmanından ayrılarak merkezi bir yapıda toplandığı tasarım katmanıdır.

ViewSet: HTTP isteklerini işleyen ve endpoint davranışlarını yöneten Django REST Framework bileşenidir.

Serializer: Veri doğrulama ve model–JSON dönüşümünü sağlayan bileşendir.

DOM (Document Object Model): HTML dokümanlarının ağaç yapısında temsil edilmesidir.

State: React bileşenlerinde kullanıcı arayüzünü belirleyen veri durumudur.

RBAC (Role-Based Access Control): Kullanıcı yetkilerinin rol bazlı olarak yönetildiği erişim kontrol modelidir.

Business Logic: Uygulamanın temel iş kurallarını içeren mantıksal katmandır.

Referanslar

- [1] Wikipedia, “JSON Web Token (JWT),” https://en.wikipedia.org/wiki/JSON_Web_Token
- [2] NIST, “Role Based Access Control (RBAC),” Sandhu et al.
- [3] Django Software Foundation, “Django Documentation,” <https://docs.djangoproject.com/>
- [4] React, “React Documentation (Learn React),” <https://react.dev/learn>
- [5] Django REST Framework, “Official Documentation,” <https://www.django-rest-framework.org/>
- [6] R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” 2000.
- [7] SQLite Consortium, “SQLite Documentation,” <https://www.sqlite.org/>
- [8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software," Addison-Wesley Professional, 1994.